

حل کننده پازل 8

محمد عرفان هومانفر
پروژه اول هوش مصنوعی

مقدمه

برای حل این پازل میتوان از روش های مختلف از جمله جستجو های آگاهانه و ناآگاهانه استفاده کرد من در این کد از سه روش که شامل دو روش ناآگاهانه و یک روش آگاهانه میباشد استفاده کردم

- ۱- روش ناآگاهانه جستجوی اول عمق (DFS)
- ۲- روش ناآگاهانه جستجوی اول سطح (BFS)
- ۳- روش آگاهانه الگوریتم A* با استفاده از فاصله منتهی

توضیحات اصلی کد

کد حل کننده پازل ۸ توسط دیزاین پترن strategy/Factory Pattern نوشته شده و از سه کلاس کلی تشکیل شده، Puzzle نشون دهنده پازل است، PuzzleInputStrategy که کلاس کلی برای روش های وارد کردن پازل توسط کاربر است. PuzzleSolver کلاس کلی برای روش های حل پازل است

برای قسمت PuzzleInputStrategy سه عدد زیرکلاس برای گرفتن ورودی از کاربر وجود دارد.

UserInputStrategy:

این با نمایش دادن محیط گرافیکی توسط کتابخانه tkinter به کاربر اجازه میدهد که پازل هشت خود را وارد کند

RandomPuzzleStrategy:

این روش با گرفتن عددی از کاربر ، به اندازه وارد شده پازل را برمیزند و به صورت رندوم تغییر میدهد

FilePuzzleStrategy:

این روش با گرفتن مسیر فایل پازل را از روی فایل میخواند

ASTARSOLVER

الگوریتم Astar یک جستجو آگاهانه است که کمترین فاصله را با توجه به دوتا آیتم حدس میزد

هزینه و تعداد استپ هایی که تا الان پیمایش شده : $g(n)$

هزینه حدس زده شده تا رسیدن به حالت فعلی درمورد ما فاصله منتهی : $h(n)$

فاصله منتهی این گونه محاسبه میشود که فاصله عمودی و افقی تا هدف را بررسی کرده و باهم جمع میکنیم تا متوجه بشیم که به طور تقریبی چه مقدار با هدف خود فاصله داریم

در کد از ساختار داده heapq استفاده میکنیم تا همیشه امیدبخش ترین نود را گسترش بدیم درحلقه تا زمانی که حالت های پیش رو که در هیپ ذخیره کردیم وجود دارد ادامه میدهیم. از اولین نود شروع میکنیم که حالت اولیه ما میباشد. ابتدا بررسی میکنیم که آیا به هدف رسیده ایم یا نه اگر نرسیده بودیم ادامه میدهیم سپس بررسی میکنیم حالت تا به الان بررسی شده است یا نه اگر شده است این حالت را رد میکنیم،

با استفاده از تابع `get_successors` تمام حالت های ممکن از حالت فعلی بدست میاوریم این تابع دو برگشت دارد یکی جهت حرکت و دیگری پازل جدیدی که به ما میده اون حرکتو میده با این تابع تمام حالت ها (نود) های ممکن را پیدا میکنیم و به هیپ اضافه میکنیم

حال تک تک این حالت های ممکن رو بررسی میکنیم اگر درون لیست حالت های دیده شده ما نبود عمل میکنیم ، ابتدا مسیر پیمایش شده را به مسیر قبلی اضافه میکنیم تا مسیر را بدست بیاریم، اندازه مسیر فعلی را با فاصله منتهن را بدست میاوریم

فاصله منتهن این گونه بدست می آید که فاصله عمودی و افقی تا فاصله عمودی و افقی هدف را برای هرکدام از آیتم ها را حساب کرده و جمع کرده و سپس این مجموع فاصله را برمیگردونیم

این فاصله منتهنی را به عنوان قسمت اولویت درون هیپ خودمون قرار میدهیم و به هیپ اضافه میکنیم

همینگونه دونه دونه پیمایش میکنیم تا به نتیجه نهایی برسیم

BFS & DFS

در پیمایش BFS یا سطحی ابتدا تمام حالت های درون لول اول را بررسی میکنیم تا به سراغ نود های بعدی برویم

در پیمایش DFS یا عمقی هر عمق را پیش میرویم تا به آخرین نود برسیم سپس به بالا برمیگردیم

ابتدا حالت اولیه پازل را به تاپل تبدیل میکنیم و به همراه یک لیست خالی از حرکت ها درون صف قرار میدهیم
ستی از نود های بازدید شده نیز تولید میکنیم

آخرین state درون صف را خارج کرده ایم بررسی میکنیم که به هدف رسیده ایم اگر نرسیده بودیم آن نود را به لیست نود های بازدید شده اضافه میکنیم و به سراغ گسترش آن نود میرویم

برای گسترش هرنود تک تک حالت هایی که از حالت فعلی میتوان بدست آورد را پیدا کرده به انتهای صف اضافه میکنیم

سپس دوباره از اول شروع میکنیم و انقدر ادامه میدهیم تا نهایتا به هدف برسیم.

در BFS ما از صف برای ذخیره سازی نود ها استفاده میکنیم، تا آخرین نودی که وارد شده اول از همه پیمایش شود ولی در DFS چرا که پیمایش عمقی داریم از ساختار داده استک استفاده میکنیم



پیچیدگی زمانی BFS

تمام حالت های ممکن را قبل از اینکه به عمق بعدی برود بررسی میشود، کوتاه ترین مسیر پیدا شده حتمی چرا که تمام حالت ها بررسی شده
تعداد شاخه ها را b در نظر میگیریم که از آنجایی که ۴ حالت برای فضای خالی جهت حرکت وجود دارد که در گوشه ها و لبه ها کمتر است به طور میانگین ۳ حالت در نظر میگیریم عمق را d در نظر میگیریم برای پازل هشت ماکسیمم ۳۱ میتواند عمق آن باشد
در بدترین حالت باید تمامی حالت های ممکن در همه لول ها بررسی شود که $O(b^d)$ میشود در پازل هشت تقریبا برابر 3^{31}

پیچیدگی مکانی BFS

از صف برای ذخیره سازی داده ها استفاده میکنیم و در بدترین حالت صف باید تمام حالت هارو در خودش نگه داری کند که معمولا زودتر به جواب میرسد و پیدا میشود همچنین تمام حالت های بررسی شده فعلی نیز نگه داری شود که میشود

$$O(b^d) + O(TotalState)$$

پیچیدگی زمانی DFS

پیچیدگی زمانی مانند BFS با این تفاوت که تا عمق پیش میرود، ممکن است سریع تر نسبت به BFS جواب برسیم اما جواب بدست آمده قطعا کوتاه ترین مسیر نمیباشد
 $O(b^d)$

پیچیدگی مکانی DFS

از پشته برای ذخیره سازی استفاده میکنیم، استک در بدترین حالت به اندازه عمق پر میشود که برابر d است
 $O(d)$

پیچیدگی زمانی A^*

اگر از روش خوبی مانند فاصله منتهی استفاده کنیم به شدت کاهش میابد در بدترین حالت در این الگوریتم برابر

$$O(b^d)$$

اما این الگوریتم بیشتر مواقع تنها بخشی از حالت هارا بررسی میکند و پیچیدگی بسیار کمتر از آن است

پیچیدگی مکانی A^*

از صف اولویت دار استفاده کردیم، در بدترین حالت ما $O(b^d)$ ذخیره میکنیم، حالت های بررسی شده را نیز نگه داری میشود که میشود

$$O(b^d) + O(TotalState)$$

